

Les Migrations en Base de Données

1. Introduction

Les **migrations de base de données** sont des scripts versionnés permettant de décrire et d'automatiser les modifications du schéma d'une base de données (ajout/suppression de colonnes, création de tables, index, etc.).

Elles sont particulièrement utiles pour :

- Suivre les changements dans le temps.
 - Répliquer les modifications dans plusieurs environnements (dev, staging, production).
 - Synchroniser les schémas entre les développeurs.
-

2. Pourquoi utiliser des migrations ?

Sans migration :

- Les développeurs modifient manuellement les schémas.
- Risque d'oublis ou d'erreurs entre les environnements.

Avec migration :

- Historique des modifications.
 - Facilité de rollback.
 - Automatisation des déploiements.
-

3. Outils courants de migration

Selon la stack :

- **Node.js** : Sequelize, Knex
 - **Python** : Alembic (SQLAlchemy)
 - **Ruby** : ActiveRecord
 - **Java** : Flyway, Liquibase
 - **PHP** : Laravel Migrations
 - **Générique** : Flyway, Liquibase
-

4. Structure d'une migration (exemple MySQL)

Prenons un exemple manuel (sans ORM), avec des scripts SQL.

Migration 001_create_users_table.sql

```
-- 001_create_users_table.sql
CREATE TABLE users (
```

```

    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(255) NOT NULL UNIQUE,
    email VARCHAR(255) NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

Migration 002_add_profile_picture_to_users.sql

```

-- 002_add_profile_picture_to_users.sql
ALTER TABLE users
ADD COLUMN profile_picture VARCHAR(255);

```

Migration 003_drop_profile_picture.sql

```

-- 003_drop_profile_picture.sql
ALTER TABLE users
DROP COLUMN profile_picture;

```

5. Versionner les migrations

Il est courant d'avoir une table spéciale **migrations** dans la base de données pour stocker les migrations déjà exécutées :

```

CREATE TABLE migrations (
    id INT AUTO_INCREMENT PRIMARY KEY,
    filename VARCHAR(255) NOT NULL,
    applied_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

Chaque fois qu'un script est exécuté, on l'insère dans cette table pour éviter de le rejouer.

6. Exemple de script PHP pour exécuter les migrations (simplifié)

```

<?php

$pdo = new PDO('mysql:host=localhost;dbname=test_db', 'root', '', [
    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION
]);

// Créer la table des migrations si elle n'existe pas
$pdo->exec("

```

```

CREATE TABLE IF NOT EXISTS migrations (
    id INT AUTO_INCREMENT PRIMARY KEY,
    filename VARCHAR(255) NOT NULL,
    applied_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)
");

// Récupérer les fichiers déjà appliqués
$stmt = $pdo->query("SELECT filename FROM migrations");
$appliedMigrations = $stmt->fetchAll(PDO::FETCH_COLUMN);

// Lister les fichiers du dossier ./migrations
$files = scandir(__DIR__ . '/migrations');
sort($files);

foreach ($files as $file) {
    if (pathinfo($file, PATHINFO_EXTENSION) === 'sql' && !in_array($file,
$appliedMigrations)) {
        $sql = file_get_contents(__DIR__ . '/migrations/' . $file);

        try {
            $pdo->beginTransaction();
            $pdo->exec($sql);
            $stmt = $pdo->prepare("INSERT INTO migrations (filename)
VALUES (:filename)");
            $stmt->execute(['filename' => $file]);
            $pdo->commit();
            echo "Applied migration: $file\n";
        } catch (Exception $e) {
            $pdo->rollBack();
            echo "Error applying migration $file: " . $e->getMessage() .
"\n";
        }
    }
}

?>

```

7. Recommandations

- **Nommer les migrations** avec un préfixe numérique ou date
(20250502_create_table_users.sql)
- **Ne pas modifier** une migration déjà appliquée (créez-en une nouvelle)
- **Tester en local** avant déploiement
- **Utiliser un outil** de migration dans les projets réels